# VALOR: Complete Guide (v2, concise notes)

**Project:** SIH26181 VALOR, *Vital and Atmospheric Logic for Offline Rescue* | **Team:** Chipsters | **Track:** MedTech and assistive wearable **What this is:** FPGA-accelerated, fully offline health companion and edge disaster monitor (ESP32-S3 + Renesas ForgeFPGA SLG47910V on a Vicharak ShrikeFi board). **Revision 2:** every number below was checked against the RTL, firmware, synthesis logs and test reports. Battery life and production cost are deliberately not estimated (see Section 14, Q29).

## How to use these notes

- **Reading order:** Sections 0 to 3 (big picture), then 4 to 6 (link, RTL, FSMs), then 8 to 9 (AI, rules). Rehearse with 12 (demo), 13 (Q&A), 14 (limits), 15 (deck fixes).
- **Two kinds of boxes:**

  👴 **Layman box** explains a hard idea in everyday words, so any teammate can explain it to a non-technical judge.

  🎤 **Say it smartly** appears wherever something is uncertain, unfinished or easy to over-claim. It tells you what to say instead of bluffing.

- Anything you cannot defend in one sentence is listed in **Section 16 (Still to confirm)**. Check those before the presentation.

## 0. Five-minute summary

**Problem.** In a disaster, mobile towers and cloud services fail, yet people still need health monitoring, and heat, smoke and storms make their condition worse. Normal wearables need the cloud, and health apps and weather tools are separate products.

**Idea.** Fuse body signals and environment signals **on the wrist**, run everything **on-device**, and warn early.

**How (two brains).**

1. **FPGA** (Verilog, 50 MHz): filters the pulse signal and detects each heartbeat in hardware, with a 20 ns hardware timestamp.
2. **ESP32-S3** (FreeRTOS, 2 cores): reads sensors, runs signal quality checks, a 619-byte INT8 neural network, clinical early-warning rules and an SOS state machine.
3. **Outputs:** OLED emergency card, a blue LED that blinks on every heartbeat (driven by the FPGA), and an offline Wi-Fi dashboard served by the board itself.

### Numbers to remember

| Item | Value |
|---|---|
| PPG sample rate | 100 Hz (one sample every 10 ms) |
| ESP32 to FPGA link | SPI Mode 0, 8-bit, 1 MHz |
| FPGA clock | 50 MHz internal oscillator (20 ns period) |
| Filter | 8-tap moving average, 35 ms delay |
| Peak detector | 4-state FSM, 2-bit binary encoding |
| Refractory period | 250 ms = 12,500,000 cycles (max 240 BPM) |
| LED pulse | 50 ms = 2,500,000 cycles |
| ForgeFPGA fit (shipped bitstream) | 515/1120 LUT5 (45.98%), 303 FFs, 105/140 CLBs, 0 BRAM, 0 PLL, 5/19 GPIOs |
| ForgeFPGA timing | WNS +7.316 ns at 50 MHz, Fmax 78.85 MHz |
| Zynq baseline | 185 LUT, 266 FF, 0 DSP, 0 BRAM, WNS +5.603 ns, Fmax 69.45 MHz |
| Neural network | 6 → 24 → 16 → 3, 619 bytes, 88.39% tier agreement with its teacher (7,000 samples) |
| MIMIC-III check | 16,387 records: accuracy 94.11%, specificity 98.19%, **sensitivity 58.18%** |
| FreeRTOS tick | 200 Hz (5 ms) |
| SOS timers | 15 s (vitals), 10 s (fall), 5 s cancel hold, 30 s auto stand-down |
| Power (measured) | 240 mA peak from USB 5 V (prototype is USB-powered) |

# 1. Problem and idea (short)

- **Cloud fails in disasters.** No network means a cloud health app has no analysis at all.
- **Body and environment are watched separately.** Fitness wearables watch the body, weather tools watch the air, and they are not combined on one device.

- **Noise causes false alarms.** Motion artifacts in pulse signals trigger alarms, so VALOR gates bad data (signal quality check) and delays escalation (confirm windows).

  👴 **Layman box: why "offline" matters.** Your smartwatch sends your data to a company's computer, which sends an answer back. In a flood or cyclone that road is cut. VALOR carries its own "doctor's rulebook" and "weather brain" in the device, so it keeps working with no signal.

  🎤 **Say it smartly.** Do not say "smartwatches are bad" or "nobody fuses both". Say: "*Consumer watches are built for fitness and rely on a connected phone or cloud. VALOR is built for the case where nothing is connected.*" The deck's "none fuse both" is an absolute claim; prefer "few products fuse both, and none work fully offline in a disaster context".

## 2. System architecture

```
SENSORS                ESP32-S3 (FreeRTOS)              FPGA (ForgeFPGA)
MAX30102 (PPG) --I2C-->  Core 0: read sensor, scale to 8 bit  --SPI-->  8-tap filter
BME280  (T/H/P) -I2C->   Core 1: SQI, HRV, SpO2, pressure     <-SPI--  peak detector FSM
MPU6050 (IMU)   -I2C->         TinyML + mNEWS2 + SOS             beat LED (50 ms)
PMS5003 (PM)    -UART->         web server (SoftAP)
GNSS module     -UART->
                  |-> OLED (I2C)   |-> Wi-Fi dashboard   |-> buzzer (confirm fitted)
```

### Data path (one heartbeat)

1. MAX30102 captures an 18-bit IR sample every 10 ms.
2. Core 0 reads it over I2C, removes the slow baseline and scales it to 8 bits (centred on 120).
3. Core 0 sends the 8-bit sample to the FPGA over SPI (MOSI).
4. FPGA filters it (8-tap average) and runs the peak detector.
5. On a confirmed beat the FPGA latches a 32-bit cycle count (the interval since the last beat), pulses the blue LED for 50 ms, and returns the result over SPI.
6. SPI returns `{beat flag, filtered[6:0]}` on MISO. After a beat, a 5-byte burst follows: `0xC3` then the 32-bit interval, most significant byte first.
7. Core 0 converts cycles to milliseconds, accepts only 30 to 250 BPM, stores it, and releases a mutex.
8. Core 1 (1 Hz) computes HR, HRV (RMSSD), SpO2, respiration, signal quality, pressure trend, runs the neural network and rules, updates the SOS state machine, and the display and web page.

## Parts

| Part | Job | Interface |
|---|---|---|
| ESP32-S3 + ForgeFPGA SLG47910V (Vicharak ShrikeFi) | compute | SPI between them |
| MAX30102 | red/IR pulse sensor (PPG), die temperature | I2C |
| BME280 | ambient temperature, humidity, pressure | I2C |
| PMS5003 | dust and smoke particles (PM1, PM2.5, PM10); the only 5 V part | UART |
| MPU6050 | 6-axis motion, fall detection | I2C |
| SSD1306 OLED | 128×64 local display | I2C |
| Quectel L89HS (u-blox NEO-M8N alternative) | satellite position | UART2 |
| Blue LED | heartbeat indicator, driven by the FPGA pin | direct |

👴 **Layman box: what is an FPGA?** A microcontroller is a person following a to-do list one step at a time. An FPGA is a box of blank electronic building blocks that you wire into your own circuit. That circuit does its job on every clock tick, in parallel, with no list to follow, so its timing never wobbles.

👴 **Layman box: what is PPG?** You shine red and infrared light into a fingertip. Each heartbeat pushes a little more blood in, which absorbs slightly more light. The sensor sees a tiny wave in the light; the peaks are your heartbeats. Comparing red and infrared absorption also gives blood oxygen (SpO2).

🎤 **Say it smartly (parts you may be asked about).** (a) The guide mentions a buzzer ("siren"), but the deck's parts list has none. Confirm it is fitted before the demo. (b) The deck's roadmap lists the IMU as "next", yet a fall-detection state machine, unit tests and one bench drop test exist. Say: "*Fall detection is working at prototype level on a breadboarded IMU; it is not yet in the final wearable build.*"

# 3. The two hardware targets

| | Xilinx Zynq-7000 (baseline) | ForgeFPGA on ShrikeFi (demo) |
|---|---|---|
| Role | golden reference, proves the RTL and timing | physical demo, low-cost target |
| Host | ARM Cortex-A9 inside the chip | ESP32-S3 |
| Link | memory-mapped AXI4-Lite bus | 4-wire SPI Mode 0 |
| Threshold | writable by software | fixed at 120 in the RTL |
| Resources | 185 LUT, 266 FF, 0 DSP, 0 BRAM | 515 LUT5, 303 FF, 0 BRAM, 0 PLL |
| Timing | WNS +5.603 ns, Fmax 69.45 MHz | WNS +7.316 ns, Fmax 78.85 MHz |
| Testbench | 6/6 self-checking checks | 13/13 self-checking checks |

The same vendor-neutral Verilog (filter, peak detector) runs on both. Zynq proves the design; ShrikeFi proves it fits a tiny, cheap FPGA. **ShrikeFi is what you show.**

🎤 **Say it smartly.** The guide quotes "about $0.50" and "under 5 mW" for the ForgeFPGA; neither figure is in the repo. Say "*a very low-cost, low-power FPGA*" unless you can cite the datasheet. The ESP32 stores the FPGA bitstream in its firmware (46,408 bytes, hash prefix `6930ba88`) and loads it into the FPGA.

# 4. Protocols and the ESP32 ↔ FPGA link

## Which bus is used where

| Bus | Used for | Why |
|---|---|---|
| I2C (shared, 2 wires, 100 kHz in firmware) | MAX30102, BME280, MPU6050, OLED | many slow sensors on two wires; each has an address |
| UART (9600 baud) | PMS5003 (UART1), GNSS (UART2) | these modules simply stream text/packets |
| SPI (4 wires, 1 MHz) | ESP32 ↔ FPGA | fast, full duplex, minimal FPGA logic |

## Why SPI (and not I2C or UART)

| | SPI (chosen) | I2C | UART |
|---|---|---|---|
| Wires | 4 (SCK, SS_n, MOSI, MISO) | 2 | 2 |
| Clock | master provides it | master provides it | none; both sides must match baud |
| Duplex | full (send and receive together) | half | full |
| Addressing | none (chip-select wire) | address + ACK on every byte | none |
| FPGA logic cost | tiny: a shift register | large: address match, ACK, open-drain | moderate: baud generator, oversampling |

SPI is a shift register: every clock tick sends one bit out and takes one bit in. The sample goes out on MOSI while the previous result comes back on MISO, so nothing is wasted.

👴 **Layman box: SPI in one picture.** Two people pass a bucket of 8 balls between them. The master taps a beat; on each tap one ball moves each way at the same time. After 8 taps both have swapped a full byte. There is no "hello" or address, just the swap.

## SPI Mode 0 and the frame

- **Mode 0:** clock idles low; data is sampled on the rising edge and changed on the falling edge. In `spi_target.v` this is hard-wired (the CPOL/CPHA parameters are declared but never used).
- **Normal frame (8 bits):** MOSI carries the sample byte; MISO returns `{beat, filtered[6:0]}`. The transfer *is* the data; there are no command bytes.
- **Burst after a beat (5 bytes):** `0xC3` (marker) + 4 bytes of the 32-bit interval, MSB first. While the burst is clocked out, the dummy MOSI bytes are blocked from entering the filter (`sample_valid = rx_valid && burst_cnt == 0`).
- **MISO shows only 7 bits** of the filtered value (bit 7 is the beat flag), so the returned waveform wraps at 128. It is diagnostic only; the beat flag and the 32-bit interval are what matter.
- **Only the IR channel** goes to the FPGA. The red channel stays in the ESP32 for SpO2.
- **Retired idea:** an early 4-bit "nibble" parallel bus was dropped before implementation (too many pins for the 19-pin package). Do not claim it exists.

## Clock domain crossing (CDC): the Verilog-teacher question

- SCK, SS_n and MOSI come from the ESP32 and are **asynchronous** to the FPGA's 50 MHz

clock.

- Each is passed through a **2-flop synchronizer** ( `sck_sync` , etc.) and edges are detected from the last two samples ( `2'b01` = rising, `2'b10` = falling). The FPGA therefore *oversamples* SCK instead of using it as a clock.
- At 1 MHz one SCK period is 50 FPGA clocks, so edges are never missed. The synchronizer also keeps a metastable value from spreading.
- **Do not** clock FPGA logic from SCK: SCK stops between bytes and breaks clocking rules.

👴 **Layman box: metastability.** If a signal changes at exactly the moment a flip-flop looks at it, the flip-flop can hesitate between 0 and 1 for a moment. Two flip-flops in a row give it a whole extra clock tick to "make up its mind" before anything else uses it.

🎤 **Say it smartly ("how fast can SPI go?").** We ran 1 MHz. The limit comes from oversampling SCK with a 50 MHz clock, so SCK must stay well below 25 MHz; we have not characterised the maximum and do not claim more than 1 MHz tested. Also note that the **deck says I2C at 400 kHz, while the firmware runs 100 kHz** (conservative for breadboard wiring). Quote 100 kHz.

# 5. FPGA and RTL

## 5.1 Why an FPGA at all?

- The ESP32 is fast. The issue is **scheduling**: FreeRTOS ticks at 200 Hz (5 ms), so software timing wobbles with task load. The FPGA's counter, filter and detector run on a fixed 50 MHz clock regardless of CPU load.
- The FPGA owns the beat decision, the interval counter and the LED, freeing Core 0.

🎤 **Say it smartly: is "20 ns" real?** The counter has **20 ns resolution**, but accuracy is limited by how regularly samples arrive: the FPGA sees one sample every 10 ms, and the ESP32 delivers it over SPI on its own schedule. So say *"20 ns resolution, with real accuracy bounded by the 100 Hz sample spacing. The benefit is that detection and interval counting do not depend on CPU task scheduling. For tighter HRV we would feed the FPGA straight from the sensor interrupt."* This is the honest, defensible version, and a sharp judge may find the gap, so own it first. The deck and demo line "eliminating jitter down to 20 ns" over-claims.

## 5.2 RTL basics used in VALOR

- **Module / ports:** a hardware block with named inputs and outputs. **Wire** = a connection; **reg** = storage (holds a value between clock edges).
- **Sequential block:** `always @(posedge clk ...)` updates registers on the clock edge using non-blocking `<=` . **Combinational logic** has no memory.
- **Parameters** make blocks reusable, e.g. `REFRACTORY_CYC = 12_500_000` .
- **Synthesis** turns Verilog into logic gates; **place-and-route** assigns them to physical spots on the chip and connects them.

👴 **Layman box: RTL.** It is a way of writing down a circuit as "what happens to each storage box on every clock tick". The tools turn that description into real wiring. It describes hardware, not a program that runs step by step.

### 5.3 Reset (be honest about this)

- No reset pin on the ForgeFPGA, so a **255-cycle power-on counter** holds everything in reset. `rst_n = (por_cnt == 255)` (a decode of the counter).
- The blocks then use `always @(posedge clk or negedge rst_n)`, i.e. an **asynchronous** reset. State updates are synchronous, but the reset is not.

🎤 **Say it smartly.** Never say "100% synchronous". Say: "*State updates are synchronous. Reset is asserted by a power-on counter and used asynchronously. We know best practice is async-assert, sync-deassert; our planned fix is to pass* `rst_n` *through two flip-flops.*" Owning the flaw before the examiner finds it scores better than being caught.

### 5.4 8-tap moving average filter

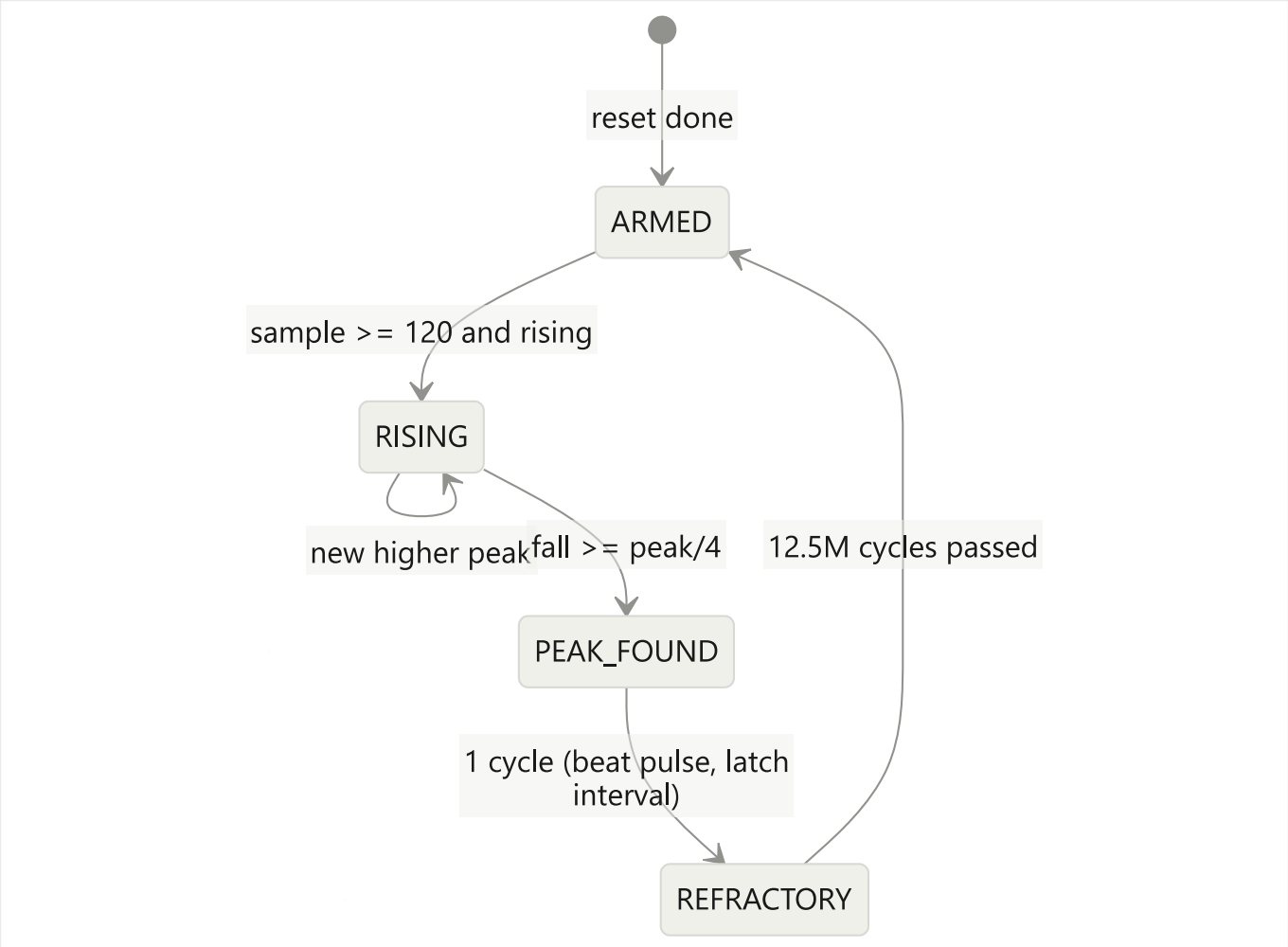
- **What:** each output is the average of the last 8 samples; it smooths sensor noise.
- **How:** a running sum: `sum_new = sum_old + x[n] - x[n-8]`. The sample leaving the window is `x[n-8]` (`shift_reg[7]`). Output = `sum >> 3`.
- **Width:** 8 samples × 8 bits needs 11 bits (8 + 3) so the sum cannot overflow.
- **Divide by 8 is free:** dividing by a power of two is just dropping 3 wires.
- **Cost:** average delay is 3.5 samples = 35 ms at 100 Hz, negligible for heart rate.

👴 **Layman box: moving average.** Instead of trusting one jumpy reading, you average the last 8 readings. Spikes shrink, the real heartbeat shape stays.

### 5.5 Peak detector FSM (the key circuit)

**Type:** registered Moore machine. The output `beat_detected` is set from the current state (`PEAK_FOUND`), not combinationally from the input. **Encoding:** 2-bit binary (2 flip-flops instead of 4 for one-hot; fine for a small FSM).

| State | Code | What it does | Moves on when |
|---|---|---|---|
| `ARMED` | 00 | waits for a pulse | sample ≥ threshold (120) **and** rising → `RISING` |
| `RISING` | 01 | tracks the highest sample ( `peak_val` ) | sample falls ≥ `peak/4` below the peak → `PEAK_FOUND` |
| `PEAK_FOUND` | 10 | one clock: beat flag, latch interval, start 250 ms timer | always → `REFRACTORY` |
| `REFRACTORY` | 11 | ignores everything (dicrotic notch, noise) | timer = 0 and a new sample → `ARMED` |




**What sequence does it detect?** Not a bit pattern. It detects a **pulse shape**: (1) a rising crossing of the level 120, (2) a tracked crest, (3) a drop of at least 25% of that crest, (4) then 250 ms of blanking.

**Why 250 ms?** The arterial pulse has a second small bump (dicrotic notch) that would be counted as a second beat. 250 ms blanking also caps the rate at 240 BPM.

**Details a Verilog examiner may probe:**

- The first crest after reset is deliberately swallowed (`first_beat_seen`) to seed the interval counter; it gives no beat flag and no LED blink.
- The weak-signal floor in `crest_fall_min = (peak > 48) ? peak>>2 : 6` can never run: arming needs a sample ⩾ 120, so the peak is always > 48. The effective rule is always `peak/4`.
- `RISING` has **no timeout**: a plateau that never drops 25% would hold the FSM there.
- The threshold is fixed at 120 (the ESP32 scales the input so the pulse is centred on 120).

🎤 **Say it smartly.** Present these as *known design limits with planned fixes*: a watchdog for `RISING`, and a threshold or baseline tracker in hardware. For weak pulses: with the 25% rule the detector needs roughly a few hundred raw counts of pulse swing (about 270 counts peak-to-peak by our back-of-envelope calculation for a symmetric pulse; verify on a real recording before quoting). Weak signals are handled upstream by the signal quality gate.

### 5.6 LED pulse stretcher

A beat is only 1 clock (20 ns) long, which the eye cannot see. A counter holds the LED on for 2.5 M cycles (50 ms). This blue LED is driven directly by an FPGA pin, which makes a good live proof that the FPGA, not software, detects the beats.

### 5.7 Resources and timing

| Resource | Used / available | Meaning |
|---|---|---|
| LUT5 (logic) | 515 / 1120 (45.98%) | filter, FSM, 32-bit counters, burst sequencer |
| Flip-flops | 303 / 1856 (16.33%) | pipeline, counters, synchronizers |
| CLBs | 105 / 140 (75%) | the real limit: wide counters occupy CLBs even when LUTs are spare |
| GPIOs | 5 / 19 | SCK, SS_n, MOSI, MISO, LED |
| BRAM / PLL / DSP | 0 / 0 / none | pure logic; the 50 MHz clock is the on-chip oscillator |

- **Slack** = how much earlier than the deadline a signal arrives. **WNS** = slack of the slowest path. WNS +7.316 ns at a 20 ns clock means the slowest path needs about 12.7 ns, so Fmax ≈ 78.85 MHz. Positive slack = timing is met with margin.
- **Why LUTs grew 342 → 363 → 362 → 535 → 515:** slope check in `ARMED`, then the 32-bit interval counter and 5-byte burst (+dummy-byte blocking), then a `peak/4` rule that rejects the dicrotic notch.
- Zynq's full-system runs show WNS +12.836 ns (1,806 endpoints) and +13.168 ns (1,826 endpoints) in two different revisions. Quote +5.603 ns as the baseline figure.

👴 **Layman box: slack.** Think of a relay race where each runner must finish before the next whistle. Slack is how many nanoseconds early the slowest runner arrives. Early (positive) is safe; late (negative) means the circuit could read garbage.

🎤 **Say it smartly.** The deck still shows the **older numbers (364 LUTs, 204 FFs, "two thirds free").** The shipped bitstream is 515 LUTs with about 54% of LUTs free, but **CLBs are 75% used**, so say "*about half the logic is free; CLB packing is the binding limit*".

### 5.8 Simulation proof

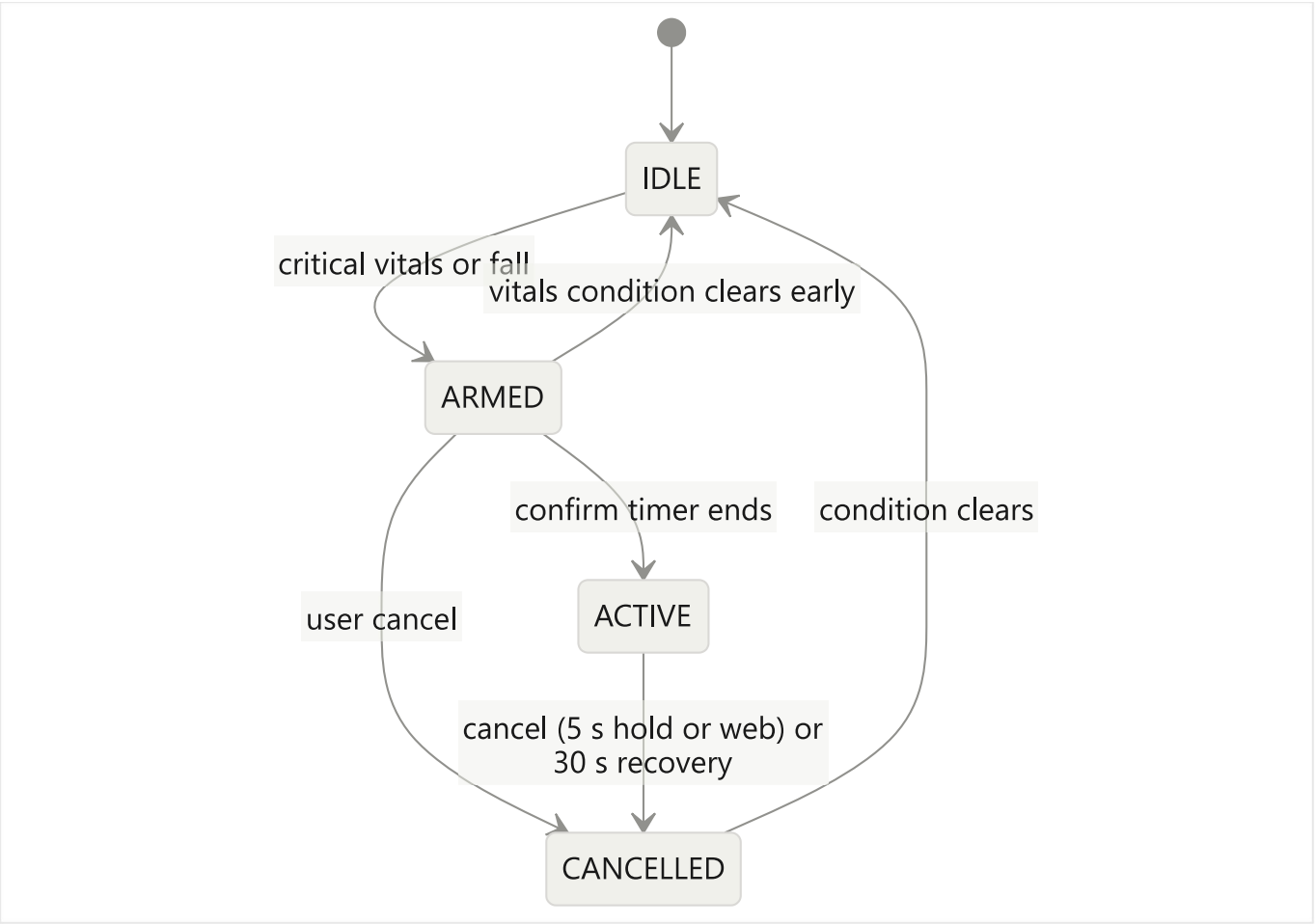
- Testbenches (run with `iverilog` + `vvp`): ShrikeFi 13/13 self-checking checks pass; Zynq 6/6 pass. You can run these live.
- The testbenches **shrink** the 250 ms refractory (500 cycles in the Zynq bench, 1800 in the ShrikeFi bench) so simulation finishes in microseconds. That is why the deck's waveform shows two beats about 66 µs apart (3279 cycles × 20 ns = 65.6 µs).

🎤 **Say it smartly.** If a judge asks about the "66 µs heartbeat": "*That is a compressed-time testbench. Real beats are around 0.8 s apart, which is about 40 million 50 MHz cycles.*" (The deck's note says "compressed time scale"; say it out loud.)

## 6. The other state machines

### 6.1 SOS (emergency) state machine: `sos.c`

| State | Meaning | Goes to |
|---|---|---|
| `IDLE` (0) | normal monitoring | `ARMED` when critical triage (finger present) or a fall is armed |
| `ARMED` (1) | timing the trigger: **15 s** for vitals, **10 s** for a fall | `IDLE` if a vitals trigger clears early; `ACTIVE` when the timer ends (or critical appears during a fall countdown); `CANCELLED` if the user cancels |
| `ACTIVE` (2) | emergency latched: OLED card, buzzer, web banner | `CANCELLED` by cancel, or after **30 s** of continuous non-critical (vitals trigger only) |
| `CANCELLED` (3) | stood down; latched so it does not re-trigger immediately | `IDLE` when the condition clears |




- **Cancel:** a 5 s button hold or the web "cancel" button (a bump cannot cancel a real emergency).
- A manual trigger (dashboard/console) goes straight to `ACTIVE`. A "contact lost" trigger is implemented but **disabled**, because a fingertip device is constantly taken off.
- **Why an FSM:** it stops one corrupt second of data from latching an emergency and guarantees a safe way out.

### 6.2 Fall-detection FSM: `fall_detect.c` (5 states)

`MONITORING` → `FREE_FALL` (acceleration < 0.60 g) → `IMPACT` (> 2.50 g within 800 ms with rotation ≥ 80 °/s) → `INACTIVITY` (still for 1.5 s) → `CONFIRMED` (arms the SOS 10 s countdown). The sequence matters: a dropped bag gives impact but the person keeps moving.

### 6.3 Not really state machines (know the difference)

- **SPI target (`spi_target.v`):** a shift register plus a 3-bit bit counter. Not an FSM; do not call it Mealy.
- **IBI burst sequencer:** a down-counter (5 → 0) that selects which byte goes out.
- **AXI4-Lite handshake (Zynq only):** address/data/response handshake flags in the AXI wrapper.

🎤 **Say it smartly.** If asked "how many FSMs?": "*Two real state machines in the design: the peak detector in RTL, and the SOS machine in firmware, plus a fall-detection FSM in firmware. The*

*SPI target and burst logic are counters and shifters.*" That is more accurate and more credible than inflating the count.

# 7. ESP32-S3 firmware

## 7.1 FreeRTOS in one paragraph

An RTOS runs several tasks that appear simultaneous; higher priority interrupts lower. Ticks are 5 ms (200 Hz). Core 0 does time-critical sensing; Core 1 does heavy analysis; Wi-Fi is pinned to Core 1 so web traffic cannot starve sampling.

| Task | Core / priority | Job |
|---|---|---|
| PPG accelerator | 0 / 5 (highest) | 100 Hz sensor read, SPI to FPGA, collect interval burst, SpO2 |
| Disaster monitor | 1 / 2 | 1 Hz fusion: SQI, HRV, mNEWS2, neural net, SOS, OLED |
| PMS5003 | 1 / 3 | UART packets, checksum |
| GPS | 1 / 3 | NMEA sentences |
| IMU | 1 / 3 | 40 Hz motion, fall FSM |

- **Mutex (`s_data_mutex`):** Core 0 writes vitals, Core 1 reads at 1 Hz. The lock keeps a reader from seeing a half-updated set of values (e.g. new interval, old SpO2).
- **I2C safety:** the driver gives up on a bus transaction after 50 ms and resets the controller instead of hanging.

👴 **Layman box: mutex.** A mutex is a "talking stick". Only the task holding the stick may change or read the shared notes, so two tasks never scribble on the same page at once.

## 7.2 MAX30102 signal path

- Runs at 100 Hz; LED current is **fixed (about 11 mA)**; the adaptive LED-current function exists but is not called.
- The 18-bit value is turned into an 8-bit stream by a **baseline tracker (about 0.64 s)** that removes the slow DC level, then gain 1/8, centred on 120 so the pulse crosses the detector threshold in both directions. **Each channel has its own baseline** (a shared one once pinned the IR byte to the rails and the FPGA saw about one beat in nine).
- SpO2 uses the red/IR ratio: `SpO2 ≈ 101.8 − 6.67 × R` (an empirical bench fit, not clinically calibrated).

- Signal quality: Karlen SQI must be ⩾ 0.70, otherwise HRV is withheld and the device stays in "acquiring".

### 7.3 Offline dashboard

- ESP32 creates an **open** Wi-Fi hotspot `VALOR-xxxx` (e.g. `VALOR-0081`, from the MAC) at `192.168.4.1`.
- The web page is stored **inside the firmware** (read-only data, not SPIFFS); the browser polls `/api/status` once a second. No internet, no app install.
- Simulated hazard profiles (Heat Wave, Smog, Flash Flood) exist in the page for demos; `launch_dashboard.bat` opens a serial-stream view.

🎤 **Say it smartly (security).** The hotspot is open by design so any rescuer can join in seconds. The trade-off: anyone in Wi-Fi range can view vitals and press cancel. Say: "*Open access is a deliberate rescue-use choice; a production version would add an authenticated cancel and optional encryption.*"

## 8. The AI layer (TinyML)

### 8.1 What it is

- **Model:** a small neural network (MLP): inputs HR, RMSSD, SpO2, temperature, humidity, PM2.5 → 24 neurons → 16 neurons → 3 outputs (heat risk, pollution risk, flood/cold-exposure risk), each in the range 0 to 1. Inputs are scaled to 0–1 using fixed ranges.
- **Size:** 144+24 + 384+16 + 48+3 = **619 values, 1 byte each = 619 bytes** (a float32 version would be about 2,476 bytes, so a 75% saving).
- **INT8 meaning:** weights are stored as 8-bit integers with a scale factor. At run time the ESP32 converts them back to floating point and uses its FPU. The sigmoid uses a software `expf`. Activations are also rounded to 8 bits between layers.
- **Why on the device:** works offline, instant, private, low power.

👴 **Layman box: neural network and INT8.** A neural network is a pile of small "weighted votes": each neuron adds up its inputs times importance numbers (weights) and passes on the result. INT8 means each importance number is stored in 1 byte instead of 4, like keeping a rounded number on a sticky note instead of writing 7 decimals. You lose a tiny bit of precision and save a lot of memory.

### 8.2 What it does and does not prove

| Check | Result |
|---|---|
| Network vs its teacher (rule-based hazard tiers), 7,000 held-out samples | FP32 88.47%, INT8 **88.39%** (quantization cost 0.08%) |

| Check | Result |
|---|---|
| INT8 vs FP32 | tier agreement 97.32%; mean error 0.00790; **max error 0.0914** |
| Speed | 0.44 µs on an x86 PC. **Not measured on the ESP32.** |
| MIMIC-III ICU feed (16,387 records) | accuracy **94.11%**, specificity 98.19%, **sensitivity 58.18%** (971 TP, 698 FN) |
| FP32 vs INT8 on that feed | 99.95% identical decisions (16,379 of 16,387) |

**Facts to know cold:**

- The network was trained on **synthetic, rule-derived data**, not on patients. The 94.11% figure checks the whole triage pipeline against an algorithmic rule on real ICU heart-rate and SpO2 values; it is *not* a clinical trial and *not* agreement with a doctor.
- In the MIMIC test the **RMSSD (HRV) column is synthetic**, because MIMIC v1.4 has no beat-to-beat data.
- **Sparsity:** 391 of 619 stored values are zero; 11 of 24 first-layer and 6 of 16 second-layer neurons are dead (all-zero inputs and a negative bias, so they always output 0). About **211 values carry signal** and the effective network is about **6 → 13 → 10 → 3.** The 619 bytes is storage, not active compute.
- An earlier **PM2.5 calibration network was removed** after bench testing: it cut readings 31% to 53% in dry air and pushed small readings to zero at high humidity, which could tell a rescuer smoky air is clean. It was replaced by the published US EPA / AirGradient correction (60 s rolling average with confidence flags).

🎤 **Say it smartly (three honest framings).**

1. **Sensitivity:** "*The benchmark shows we are conservative on false alarms (98% specificity) but miss about 42% of rule-defined critical cases. That is why hard clinical rules override the network, and why we call VALOR early-warning screening, not diagnosis.*" Quote sensitivity yourself; do not let 94.11% stand alone.
2. **Synthetic training:** "*The risk network distils our rule engine into a tiny model that runs without cloud. It is a prototype; field data from wrist devices is the next step.*"
3. **Dead neurons:** "*Regularisation left many neurons unused. It still fits in 619 bytes, and it can be retrained smaller or pruned.*"

## 9. Clinical and disaster rules (deterministic)

**Hybrid safety envelope:** the network gives smooth risk scores; fixed rules give hard limits that **always win**.

### 9.1 mNEWS2 (modified NEWS2)

NEWS2 is the UK Royal College of Physicians early-warning score. VALOR scores what a fingertip device can measure: pulse, SpO2, respiration rate (temperature and blood pressure are not reliable here, and consciousness is assumed alert unless a fall is confirmed). Bands: 0–4 low, 5–6 medium, ≥ 7 high; any single parameter scoring 3 is a red flag.

### 9.2 Hard overrides (force CRITICAL)

SpO2 ≤ 88%, heart rate ≥ 150 BPM, mNEWS2 ≥ 7, or barometric fall ≤ −2.0 hPa/hr.

### 9.3 Hazard thresholds

| Hazard | Rule |
|---|---|
| **Cyclone** | 180-minute least-squares **rate of pressure fall**: ≤ −0.5 hPa/hr moderate, ≤ −1.0 high (gale), ≤ −2.0 critical; ignored unless the drop is at least 0.5 hPa (noise floor) |
| **Heat** | Heat index (NOAA Rothfusz regression, capped at 58 °C): ≥ 35 caution, ≥ 40 moderate (or HR ≥ 95 / RMSSD ≤ 35), ≥ 45 high (HR ≥ 110 / RMSSD ≤ 20), ≥ 54 critical (HR ≥ 130 / RMSSD ≤ 10) |
| **Smoke** | PM2.5 ≥ 35 caution, ≥ 75 moderate, ≥ 150 high, ≥ 300 critical (µg/m³), or SpO2 ≤ 96/94/92/88%, or HR ≥ 100/120; hazardous AQI also sets a risk floor |
| **Cross-domain guard** | e.g. a heat alert is suppressed when the ambient temperature is cold (stops silly alarms) |

👴 **Layman box: why track pressure *falling*.** Air pressure depends on altitude (a hill town always reads lower). A storm approaching makes it *drop quickly* wherever you are. So the device watches the speed of the drop, not the absolute number.

🎤 **Say it smartly.**

- **35 µg/m³ is the US EPA 24-hour standard**, not the WHO limit (the WHO 2021 guideline is 15). Do not call it a WHO limit.
- **"Flood detect engine"** (deck): there is no water or rain sensor. The third output is a *flood/cold-exposure risk index* inferred from humidity, temperature and physiology. Say *"flood and cold-exposure risk"*, not "flood detection".
- The Rothfusz heat-index formula is only valid in warm, humid conditions, which is why the cold guard exists.
- The guide lists temperature among mNEWS2 inputs but also says it scores 0 points; **confirm in** `clinical_vitals_engine.c` which is true before presenting (Section 16).

## 10. Sensors and skin temperature

- **Why no body temperature?** The TI TMP117 skin probe did not respond on the bench (I2C bench report, 21 Oct 2026: reads 0xFFFF, bus healthy at 3.18 V, other 4 devices answered). Likely a defective breakout; root cause not isolated.
- **Why not use the MAX30102's own die temperature as body temperature?** Its LEDs and the board heat the chip, so it reads well above room temperature even with no finger (your notes record about 38.6–41.5 °C idle), which would give every person a fever.
- **What VALOR does instead (`skin_estimate.c`):** it tracks the idle die temperature, freezes it when a finger lands, and computes the *change*: `T_skin = b0 + b1·(T_die_pred − T_die_baseline) + b2·(T_amb − 27.5)` with b0 = 34.50 °C, b1 = 1.05, b2 = 0.15 and a 4 s thermal lead filter. It publishes after 3 s of contact, is "settled" after at least 15 s with slope ≤ 0.025 °C/s, and is valid only in 31–42 °C. It is shown on screen but is **not** used in mNEWS2.

  🎤 **Say it smartly.** Do not say "accurate skin temperature". Say: "*It is an estimate from a thermal model with nominal coefficients, not calibrated against a reference thermometer. The 35.7 °C figure is mostly the 34.5 °C baseline constant plus a small contact step, so we show it as indicative only and keep it out of triage. Calibration against a clinical thermometer is the next step.*" And do not claim "we deleted the die-temperature read": the driver reads it and the estimator uses it as an input.

## 11. Location and connectivity

- **GNSS hardware and code exist:** Quectel L89HS on UART2 (u-blox NEO-M8N as alternative), bench-wired and tested (UAT report); the NMEA parser has 12/12 unit tests.
- **Indoors there is no fix** (satellite signals are heavily attenuated by concrete and debris). VALOR then shows a **location text saved in flash** (e.g. "Disaster Shelter 4, Ward B"). A rescuer can change it from the phone dashboard (local HTTP POST, saved to flash).
- A room name tells a medic more than coordinates in a multi-storey shelter.

  🎤 **Say it smartly.** In a hall during judging, GNSS will say "NO FIX". Lead with that: "*This is the real-world case: no sky view, so the stored location is shown.*"

## 12. Demo script (about 7 minutes)

| Act | Do | Say (key idea) |
|---|---|---|
| 1. Boot (1 min) | hold up board, plug in USB; OLED shows "not assessed" until a finger is placed | "Offline dual-brain wearable. It refuses to grade a patient it has |

| Act | Do | Say (key idea) |
|---|---|---|
| | | not measured." |
| 2. Live pulse (1.5 min) | finger on MAX30102; blue LED blinks per beat; OLED shows HR and SpO2; optional serial oscilloscope | "This LED is driven by the FPGA pin. Detection runs in hardware, independent of CPU load." |
| 3. Hazards (1.5 min) | cup hand over BME280, show pressure trend and PM2.5 card | "It watches how fast pressure falls, not the absolute value." |
| 4. Fall + SOS (1.5 min) | drop the board about 15 cm onto a notebook, keep still 1.5 s; OLED shows fall, 10 s countdown, then the emergency card | "Free-fall, impact, stillness. The countdown lets a wearer cancel a false alarm." |
| 5. Rescuer phone (1.5 min) | join `VALOR-0081`, open `192.168.4.1`, show banner, tap cancel | "No router, no app. The wearable is its own web server." |

**If it fails**

- **No finger detected:** room light is saturating the sensor. Cup a hand over it.
- **OLED black:** check I2C jumpers and 3.3 V; power-cycle.
- **Phone will not connect:** turn mobile data off so it does not drop a Wi-Fi network with no internet.
- **Hardware dead:** open the dashboard HTML and use a simulated profile, then run the testbench live with `iverilog`.

🎤 **Say it smartly.** Replace "eliminating jitter to 20 ns" with "20 ns hardware timestamps". Replace "blinks precisely on every heartbeat" with "blinks on each detected beat" (we have not measured against a reference). The first beat after a reset is intentionally skipped.

## 13. Judge Q&A (30 questions)

**Concept**

**Q1. Why two chips instead of only the ESP32?** The FPGA runs the filter, peak detector and interval counter on a fixed 50 MHz clock regardless of CPU load, and frees Core 0. Software timing wobbles with a 5 ms tick. *Don't say:* "the ESP32 is too slow."

**Q2. How is VALOR different from a smartwatch?** Watches target fitness and rely on a connected phone or cloud. VALOR fuses body and environment signals and runs fully offline for disasters. *Don't say:* "smartwatches are bad."

**Q3. Which hardware is your product, Zynq or ShrikeFi?** ShrikeFi is the demo. Zynq is the golden reference that proved the RTL and timing (WNS +5.603 ns). The same Verilog runs on both.

**Q4. Does it need internet or an app?** No. The ESP32 hosts its own Wi-Fi hotspot and serves the page from its flash; open `192.168.4.1`.

**FPGA and RTL**

**Q5. Moore or Mealy?** Registered Moore: `beat_detected` depends on the state `PEAK_FOUND`, not directly on the input. *Don't say:* "Mealy because inputs cause transitions" (transitions depend on inputs in every FSM; the question is about outputs).

**Q6. Why binary encoding?** Two flip-flops instead of four for a four-state machine; speed difference is negligible at 50 MHz. *Don't say:* "binary is faster."

**Q7. What sequence does it detect?** A pulse shape, not bits: rising crossing of 120, crest tracking, a ≥25% fall, then 250 ms blanking.

**Q8. Why a 250 ms refractory period?** To skip the dicrotic notch (a second small bump in each pulse) and cap the rate at 240 BPM. It is 12,500,000 cycles at 50 MHz.

**Q9. Is the design synchronous?** State updates are; reset is asserted by a power-on counter and used asynchronously (planned fix: two-flop reset synchronizer).

**Q10. Why 8 taps, and why is dividing by 8 free?** 8 is a power of two, so dividing is just dropping 3 bits; the window (80 ms) smooths noise without much delay. The sum needs 11 bits.

**Q11. What is WNS and what is your margin?** Slack of the slowest path. WNS +7.316 ns at 20 ns on ForgeFPGA (Fmax 78.85 MHz), zero failing endpoints. Negative slack means a circuit that can fail.

**Q12. Why did the LUT count rise from 342 to 515, and why are CLBs at 75%?** We added the 32-bit interval counter and the 5-byte burst, and a better crest rule. Wide counters consume CLBs even when LUTs are spare, so CLBs are the binding limit.

**Q13. Is the 20 ns timestamp real?** It is real resolution; accuracy is bounded by the 10 ms sample spacing and SPI delivery timing. The gain is that detection does not depend on CPU scheduling. (See Section 5.1.)

**Q14. What can the detector miss?** Weak pulses (below roughly a few hundred raw counts of swing), a plateau that never falls 25% (no timeout yet), and the very first beat after reset. Upstream signal quality gating and a planned watchdog address these.

**SPI link**

**Q15. Why SPI and not I2C or UART?** Full duplex, master-supplied clock, no addressing, and a shift register is all the FPGA needs. I2C needs addressing and ACK logic and is half duplex; UART has no clock and needs baud matching.

**Q16. How do you handle the clock mismatch?** SCK is asynchronous to the 50 MHz FPGA clock, so it is passed through two flip-flops and edges are detected (oversampling). At 1 MHz that is 50 FPGA clocks per SCK period.

**Q17. What if SPI runs too fast or a bit is corrupted?** Too fast and edges can be missed; we run 1 MHz and have not characterised the maximum. A burst starts with marker `0xC3` and the beat flag is repeated every transfer; a stricter checksum is future work.

**Q18. What does the SPI frame contain?** MOSI: sample byte. MISO: beat flag plus 7 bits of filtered data. After a beat, a 5-byte burst: `0xC3` + 32-bit interval.

**Sensors**

**Q19. Your TMP117 failed. What now?** We documented it (bus healthy, device silent) and show a labelled, uncalibrated skin estimate that does not feed triage. Next step is replacing the probe.

**Q20. Why not use the MAX30102 die temperature as body temperature?** Self-heating makes it read high with no finger; we use only its change after contact, as one input to an estimate.

**Q21. How do you handle motion artifacts and skin tone?** A signal quality index (≥ 0.70) withholds HRV and clinical scores when the waveform is bad. Skin tone and perfusion change signal strength; LED current is fixed in this build, so weak signals are gated rather than boosted. *Don't say:* "skin colour doesn't matter."

**AI and clinical**

**Q22. Why not a bigger model or the cloud?** No network in a disaster, tiny memory, low power. For a handful of numeric inputs a small network is enough, and rules guarantee safety.

**Q23. Why are many neurons dead?** Training regularisation left 11 + 6 neurons unused; about 211 values carry signal. It still fits in 619 bytes and can be retrained smaller.

**Q24. What data trained the model?** Synthetic, rule-derived vitals. Real ICU data (MIMIC-III) was used to test the whole triage pipeline, not to train the network.

**Q25. 94.11% accuracy, yet sensitivity 58%?** The benchmark is agreement with an algorithmic rule on 16,387 ICU records (HRV synthetic). We are strong on avoiding false alarms (98% specificity) but miss about 42% of rule-defined critical cases, so rules override the network and we call it screening. *Don't say:* "94% accurate replacing doctors."

**Q26. Is it a diagnosis? How far from regulatory approval?** No. It is an early-warning triage prototype. Approval would need ISO 13485, IEC 60601-1, prospective clinical trials and a medical-grade enclosure.

**Q27. Why do rules override the AI, and why pressure fall rate?** Neural networks can behave unpredictably out of range; deterministic limits must stand. Absolute pressure

depends on altitude; a fast fall indicates an approaching storm anywhere.

**Firmware and limits**

**Q28. Why pin tasks to cores and use a mutex?** Core 0 keeps 100 Hz sampling undisturbed; Wi-Fi is pinned to Core 1. The mutex prevents Core 1 from reading a half-updated set of vitals.

**Q29. Cost and battery?** Prototype parts total INR 3,200 (deck, retail with GST, checked 1 Oct 2026). The prototype is USB-powered and draws 240 mA peak at 5 V, measured, including the PMS5003 fan. We have not costed volume production or sized a battery; that is on the roadmap.

**Q30. How reliable is fall detection?** 16 unit tests plus one physical drop trial (n = 1) at breadboard level. A 10 s cancel window is the safeguard. It is a proof of concept, not a validated detector. *Don't say:* "100% accurate."

## 14. Honest limitations

1. **Not a medical device:** no clinical trials, no regulatory approval; early-warning triage only.
2. **No validated body temperature:** TMP117 failed; the skin value is an uncalibrated estimate.
3. **Training and test data:** the network trained on synthetic data; MIMIC HRV is synthetic; benchmark sensitivity is 58%.
4. **Fingertip PPG:** motion corrupts it, so vitals are withheld when signal quality is low.
5. **USB power only:** no battery or power-management in the prototype.
6. **SpO2 calibration:** empirical bench fit, not tested in a blood-gas laboratory.
7. **PM2.5 calibration:** inherits the published EPA / AirGradient correction; no local reference monitor.
8. **Open Wi-Fi:** anyone in range can view vitals or press cancel.
9. **Detector limits:** fixed threshold 120, no timeout in `RISING`, first beat skipped, weak pulses missed.
10. **Fall detection:** one physical trial; IMU path is prototype-level.
11. **Unmeasured:** ESP32 inference time and the dashboard frame rate (60 FPS is a design target).

    Each item has a next step: calibrate against a reference thermometer, retrain on field data, add battery and charging, add a reset synchronizer and watchdog, add an authenticated cancel, and run multi-trial fall tests.

## 15. Fix these in the deck (old numbers and over-claims)

| Deck says | Reality (use this) |
|---|---|
| 364 LUTs, 204 FFs, 73 CLBs, "two thirds free" | **515 LUT5, 303 FFs, 105 CLBs**; about 54% of LUTs free, 75% of CLBs used |
| Power slide: "FPGA (ICE40)" | Renesas ForgeFPGA SLG47910V |
| Pin map: I2C 400 kHz | firmware runs 100 kHz |
| "Flood detect engine" | flood/cold-exposure risk index (no water sensor) |
| "Integer-only math" (firmware slide) | float math on dequantized INT8 weights |
| 619 bytes as "RAM" (summary slide) | 619 bytes of **flash** (model storage) |
| "Adaptive peak threshold" (simulation slide) | adaptive in the Zynq testbench; fixed at 120 on ShrikeFi |
| IMU listed under "Next" | fall FSM and one bench trial already exist; say "prototype level" |
| "Blinks precisely" / "none fuse both" | "blinks on each detected beat" / "few products fuse both offline" |
| BOM cells `[_1__]` and similar | replace with actual unit prices and quantities |
| "94.11%" shown alone | show sensitivity 58.18% and specificity 98.19% with it |

## 16. Still to confirm in your repo (before the presentation)

1. **mNEWS2 temperature:** is temperature scored or 0 points in `clinical_vitals_engine.c`? (The guide says both.)
2. **HRV settle window:** the notes say both 30 valid beats and 240 valid beats. Check the constant before quoting.
3. **Idle die temperature on the bench:** 38.6–41.5 °C in one note, 32–34 °C in another; use your logged value.
4. **Buzzer:** fitted or not? The demo mentions a siren.
5. **ESP32 inference time:** measure with a timer; only the 0.44 µs x86 figure exists.

6. **Cause of the 698 missed critical cases** in the MIMIC test (see the AI audit report), so you can explain the 58% sensitivity.
7. **98 patients:** recount from the MIMIC feed so you can reproduce it.
8. **Datasheet claims:** FPGA cost and power figures if you want to quote them.

## 17. Glossary

| Term | Plain meaning |
|---|---|
| FPGA | chip of blank logic blocks wired into your own circuit |
| RTL | Verilog description of registers and the logic between them |
| LUT | tiny lookup table; the basic logic element in an FPGA |
| CLB | a group of 8 LUTs plus flip-flops |
| FF (flip-flop) | 1-bit storage updated on a clock edge |
| BRAM / DSP | built-in memory blocks / multiplier blocks (we use none) |
| FSM | state machine: a circuit that moves between a few named states |
| Moore / Mealy | FSM whose outputs depend on state only / on state and input |
| Refractory period | blanking time after a beat during which new peaks are ignored |
| Dicrotic notch | the small second bump in each arterial pulse |
| IBI | inter-beat interval: time between heartbeats |
| HRV / RMSSD | variation of beat intervals / its standard summary number |
| PPG | light-based pulse measurement |
| SpO2 | blood oxygen saturation |
| SQI | signal quality index |
| SPI / I2C / UART | three ways chips talk (see Section 4) |
| MOSI / MISO | master-out-slave-in / master-in-slave-out data wires |
| CDC / metastability | crossing between clock domains / unstable flip-flop state |

| Term | Plain meaning |
|---|---|
| Synchronizer | two flip-flops in a row that settle a signal from another clock |
| WNS / slack / Fmax | slowest-path margin / timing margin / highest clock that still works |
| POR | power-on reset |
| FreeRTOS / mutex | small real-time OS / lock for shared data |
| TinyML / INT8 | small on-device model / 8-bit stored numbers |
| MLP | the simple layered neural network used here |
| NEWS2 | UK early-warning score for deteriorating patients |
| Heat index | "feels like" temperature from heat and humidity |
| SoftAP | the board acting as its own Wi-Fi hotspot |
| NVS | flash storage for small settings (used for location text) |
| GNSS / NMEA | satellite positioning / its text output format |

## 18. Who should be able to explain what

| Role (from the demo runbook) | Own these sections |
|---|---|
| **Live demo and sensors** | 0–2, 4 (buses), 10, 11, 12 |
| **RTL and digital design** | 3, 4, 5, 6, 15 |
| **AI, clinical logic and firmware** | 7, 8, 9, 13 (Q22–Q30), 14 |

The team has six members; the other three should read Sections 0, 4, 5.5 and 13, so every person can answer the top questions.

## Appendix: quick pin and address reference (only if asked)

| Signal | ESP32-S3 | Other end |
|---|---|---|
| I2C SDA / SCL | GPIO1 / GPIO2 | OLED 0x3C, MAX30102 0x57, BME280 0x76, MPU6050 0x68 |
| UART1 RX (PMS5003) | GPIO14 | PMS5003 TX (TX line GPIO18 optional) |
| UART2 RX / TX (GNSS) | GPIO16 / GPIO17 | L89HS |
| SPI SCK / SS / MOSI / MISO | GPIO12 / 10 / 11 / 13 | FPGA PIN_16 / 17 / 18 / 19 |
| FPGA enable / power control | GPIO8 / GPIO9 | board-level, used for FPGA loading (not in the FPGA RTL) |
| Blue beat LED | n/a | FPGA PIN_7 |